Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) es un paradigma de diseño de software que organiza el código alrededor de **entidades del mundo real** (llamadas objetos) que combinan datos y comportamiento, en lugar de mantener variables y funciones separadas.

1. El Cambio de Mentalidad: Estructurado vs. POO

En la programación estructurada tradicional, los datos y la lógica de negocio se gestionan de forma independiente:

```
# --- ENFOQUE ESTRUCTURADO ---
jugador_nombre = "Alex"
jugador_vida = 100

def recibir_dano(vida_actual, cantidad):
    return max(0, vida_actual - cantidad)

jugador_vida = recibir_dano(jugador_vida, 25)
```

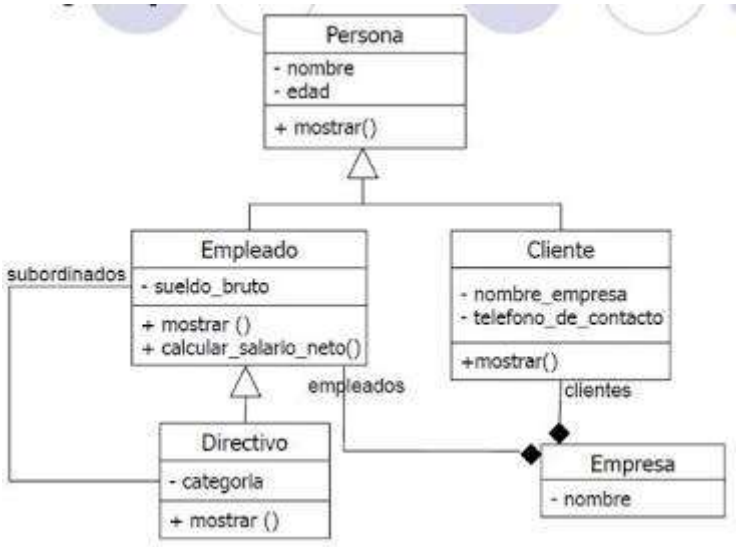
En POO, el objeto encapsula **quién es** (sus datos) y **qué puede hacer** (sus métodos):

```
# --- ENFOQUE ORIENTADO A OBJETOS ---
jugador1 = Jugador("Alex", 100)
jugador1.recibir_dano(25)  # El objeto modifica su propio estado interno
```

2. Conceptos Fundamentales

Concepto	Analogía	Definición
Clase	El plano de construcción de una casa	Plantilla o molde que define qué datos y acciones tendrá un tipo de entidad.
Objeto (Instancia)	La casa física construida con el plano	Un ejemplar concreto creado a partir de la clase, asentado en memoria.
Atributo	Color de las paredes, número de cuartos	Variables internas que almacenan el estado del objeto.

Concepto	Analogía	Definición
Método	Abrir puerta, encender luces	Funciones declaradas dentro de la clase que operan sobre sus atributos.



Estructura de clases y jerarquías en UML. Fuente: Paradigma de la programación orientada a objetos

3. Anatomía de una Clase en Python

Para construir una clase en Python utilizas la palabra reservada class, el método constructor especial __init__, y la referencia explícita self.

```python
class CuentaBancaria:
    # Método Constructor: Se ejecuta automáticamente al instanciar el objeto
    def __init__(self, titular: str, saldo_inicial: float = 0.0):
        # Atributos de instancia (datos del objeto)
        self.titular = titular
        self.saldo = saldo_inicial

    # Método de instancia (comportamiento)
    def depositar(self, monto: float) -> None:
        if monto > 0:
            self.saldo += monto
            print(f"Depósito de ${monto} exitoso. Nuevo saldo: ${self.saldo}")
        else:
            print("El monto debe ser positivo.")
```

```python
    def retirar(self, monto: float) -> None:
        if 0 < monto <= self.saldo:
            self.saldo -= monto
            print(f"Retiro de ${monto} exitoso. Saldo restante: ${self.saldo}")
        else:
            print("Fondos insuficientes o monto inválido.")

# --- Instanciación y uso ---
cuenta_ana = CuentaBancaria("Ana López", 500.0)
cuenta_ana.depositar(200)   # Saldo: 700.0
cuenta_ana.retirar(100)     # Saldo: 600.0
```

¿Qué es self y por qué es obligatorio?

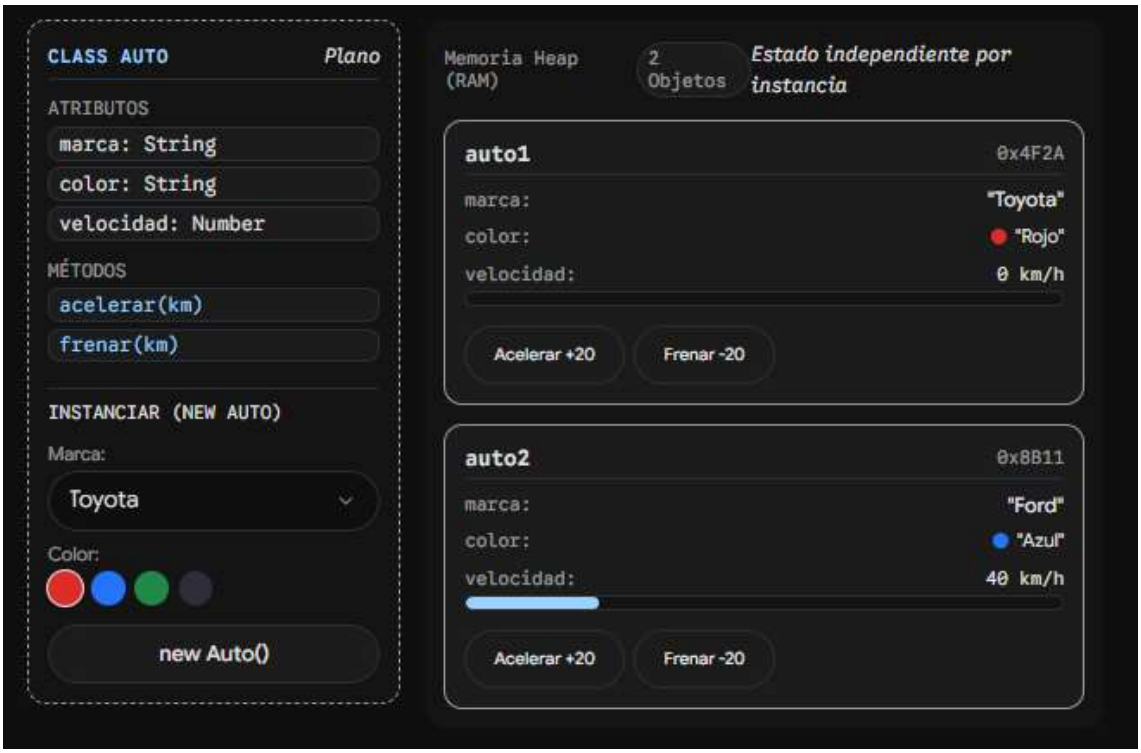
self hace referencia a la **instancia concreta** que está ejecutando el código en ese instante. Cuando escribes cuenta_ana.depositar(200), Python traduce internamente esa llamada a CuentaBancaria.depositar(cuenta_ana, 200). self le permite al método saber sobre cuál objeto en memoria debe actuar.

4. Simulador Interactivo: De Clase a Instancias en Memoria

Para comprender cómo conviven la clase y múltiples instancias independientes en memoria, interactúa con el siguiente simulador:

Clase vs. Instancias en Memoria

Una Clase es el plano estático; los Objetos son instancias independientes asignadas en la memoria Heap.



5. Los 4 Pilares Fundamental de POO

A. Encapsulamiento

Consiste en ocultar la estructura interna de los datos para evitar modificaciones directas no autorizadas. En Python se usa la convención de guion bajo (_atributo) y decoradores @property.

```python
class Personaje:
    def __init__(self, nombre: str, vida_maxima: int):
        self.nombre = nombre
        self._vida = vida_maxima  # Atributo protegido

    @property
    def vida((self) -> int:
        """Getter: Permite consultar la vida sin modificarla directamente."""
        return self._vida

    def recibir_ataque(self, dano: int) -> None:
        """Modifica la vida aplicando las reglas de negocio del personaje."""
        self._vida = max(0, self._vida - dano)
```

B. Herencia

Permite definir una clase hija que reutiliza atributos y métodos de una clase padre, reduciendo la duplicación de código.

```python
class Vehiculo:
    def __init__(self, marca: str, modelo: str):
        self.marca = marca
        self.modelo = modelo

    def encender(self):
        print(f"{self.marca} {self.modelo} encendido.")

# Moto hereda de Vehiculo
class Moto(Vehiculo):
    def __init__(self, marca: str, modelo: str, cilindrada: int):
        super().__init__(marca, modelo)  # Llama al constructor del padre
        self.cilindrada = cilindrada

    def hacer_wheelie(self):
        print(f"La moto {self.modelo} levantó la rueda.")
```

C. Polimorfismo

Permite tratar objetos de distintas clases mediante una misma interfaz o nombre de método, ejecutando comportamientos diferentes según el tipo de objeto.

```python
class Perro:
    def hablar(self):
        return "¡Guau!"

class Gato:
    def hablar(self):
        return "¡Miau!"

# Ambas clases responden al mismo método
animales = [Perro(), Gato()]
for animal in animales:
    print(animal.hablar())
```

D. Abstracción

Esconde los detalles complejos de implementación y expone únicamente los métodos esenciales para el usuario del objeto.

```python
class Lavadora:
    def lavar(self) -> None:
        # El usuario solo invoca lavar(), sin preocuparse por la secuencia interna
        self._llenar_agua()
        self._mezclar_jabon()
        self._centrifugar()
        print("Ciclo de lavado completado.")

    def _llenar_agua(self): pass
    def _mezclar_jabon(self): pass
    def _centrifugar(self): pass
```

Asociación

La **Asociación** describe la forma en que dos o más clases independientes se relacionan y colaboran entre sí para cumplir una función dentro del sistema.

A diferencia de la **Herencia** (que responde a la relación *"es un"*), la Asociación responde a relaciones del tipo *"usa un"* o *"tiene un"*.

1. Tipos de Asociación

El nivel de dependencia y el control del ciclo de vida entre los objetos determinan los 3 tipos de asociación:

A. Asociación Simple (Uso / Colaboración)

Los objetos interactúan enviándose mensajes o invocando métodos entre sí, pero **ninguno es dueño del otro**. Cada uno maneja su propio ciclo de vida de forma completamente independiente.

- **Ejemplo:** Un Mecánico usa Herramienta. El mecánico no es "dueño" del objeto herramienta, simplemente interactúa con él.

```python
class Herramienta:
    def __init__(self, nombre: str):
        self.nombre = nombre

    def usar(self):
        print(f"Usando la herramienta: {self.nombre}")

class Mecanico:
    def __init__(self, nombre: str):
        self.nombre = nombre

    # La herramienta se pasa como argumento (Asociación temporal)
    def trabajar_con(self, herramienta: Herramienta):
        print(f"{self.nombre} está trabajando...")
        herramienta.usar()
```

B. Agregación (Contención Débil — "Tiene un")

Un objeto contenedor agrupa o contiene a otros objetos. Existe una relación de pertenencia, pero **si el contenedor se destruye, los objetos contenidos continúan existiendo**.

- **Criterio clave:** Los objetos contenidos **se crean fuera** del contenedor y se le pasan como referencia.

```python
class Estudiante:
    def __init__(self, nombre: str):
        self.nombre = nombre

class Universidad:
    def __init__(self, nombre: str):
        self.nombre = nombre
        self.estudiantes: list[Estudiante] = []  # Colección agregada

    def inscribir(self, estudiante: Estudiante):
        self.estudiantes.append(estudiante)
```

```python
# Los estudiantes existen independientemente
e1 = Estudiante("Lucas")
e2 = Estudiante("María")

unsl = Universidad("Universidad Nacional")
unsl.inscribir(e1)
unsl.inscribir(e2)

# Si eliminamos la universidad, e1 y e2 siguen existiendo en memoria
del unsl
print(e1.nombre)  # Imprime: Lucas
```

C. Composición (Contención Fuerte — "Es parte de")

Un objeto está compuesto por subobjetos de los cuales **es dueño absoluto**. El ciclo de vida de las partes está acoplado al del contenedor: **si el contenedor se destruye, sus partes se destruyen con él**.

- **Criterio clave:** Los subobjetos **se instancian dentro** del constructor del contenedor.

```python
class Motor:
    def __init__(self, cv: int):
        self.cv = cv

class Auto:
    def __init__(self, marca: str, potencia_cv: int):
        self.marca = marca
        # El motor nace dentro del auto (Composición)
        self.motor = Motor(potencia_cv)

# Al crear el Auto, se crea su Motor automáticamente
mi_auto = Auto("Ford", 150)

# Si el auto se destruye, el motor asociado deja de existir
del mi_auto
```

2. Resumen Comparativo de Asociaciones

Tipo	Relación	Pertenencia	Si se destruye el contenedor...	Implementación habitual
Asociación Simple	Usa un	Ninguna	No aplica	Se pasa como parámetro en métodos
Agregación	Tiene un	Débil	Los objetos contenidos **sobreviven**	Se reciben objetos ya creados en __init__ o métodos
Composición	Es parte de	Fuerte	Los objetos contenidos **se destruyen**	Se instancian dentro del __init__

3. Cardinalidad (Multiplicidad)

La cardinalidad define **cuántas instancias** de una clase se pueden asociar con **una instancia** de otra clase.

1 : 1 (Uno a Uno)
1 : N (Uno a Muchos) o N: 1 (Muchos a Uno)
N : M (Muchos a Muchos)

A. Cardinalidad 1 a 1 (1:1)

Un objeto de la Clase A se relaciona con un único objeto de la Clase B.

```python
class Pasaporte:
    def __init__(self, numero: str):
        self.numero = numero

class Ciudadano:
    def __init__(self, nombre: str, pasaporte: Pasaporte):
        self.nombre = nombre
        self.pasaporte = pasaporte  # Exactamente 1 pasaporte por ciudadano
```

B. Cardinalidad 1 a Muchos (1:N)

Un objeto de la Clase A contiene o administra múltiples objetos de la Clase B, pero cada objeto B pertenece a un solo A.

```
class Departamento:
    def __init__(self, nombre: str):
        self.nombre = nombre
        self.empleados: list[Empleado] = []  # 1 departamento -> N empleados

    def agregar_empleado(self, empleado: Empleado):
        self.empleados.append(empleado)
```

C. Cardinalidad Muchos a Muchos (N:M)

Múltiples objetos de la Clase A se relacionan con múltiples objetos de la Clase B. En código se modela manteniendo listas cruzadas en ambas clases.

```
class Curso:
    def __init__(self, nombre: str):
        self.nombre = nombre
        self.estudiantes: list['Alumno'] = []

class Alumno:
    def __init__(self, nombre: str):
        self.nombre = nombre
        self.cursos: list[Curso] = []

    def inscribir_curso(self, curso: Curso):
        if curso not in self.cursos:
            self.cursos.append(curso)
            curso.estudiantes.append(self)  # Relación bidireccional N:M
```

4. Ejemplo Integrador en Python

Este ejemplo reúne los conceptos de **Agregación**, **Composición** y **Cardinalidad 1:N / N:M** en un mismo sistema bancario:

```
class TarjetaDebito:
    """Relación 1:1 por Composición con CuentaBancaria"""
    def __init__(self, numero: str):
        self.numero = numero

class CuentaBancaria:
    """1 Cuenta contiene 1 Tarjeta (Composición) y pertenece a N Clientes
(N:M)"""
```

```python
    def __init__(self, cbu: str, numero_tarjeta: str):
        self.cbu = cbu
        # Composición: la tarjeta nace y muere con la cuenta
        self.tarjeta = TarjetaDebito(numero_tarjeta)

class Cliente:
    """1 Cliente puede tener N Cuentas (Agregación: 1:N)"""
    def __init__(self, nombre: str, dni: str):
        self.nombre = nombre
        self.dni = dni
        self.cuentas: list[CuentaBancaria] = []  # Agregación (1:N)

    def asociar_cuenta(self, cuenta: CuentaBancaria):
        if cuenta not in self.cuentas:
            self.cuentas.append(cuenta)

# --- Pruebas del sistema ---
# 1. Creamos clientes (existen por sí solos)
cliente_carlos = Cliente("Carlos Pérez", "30111222")

# 2. Creamos una cuenta (su tarjeta se crea internamente por Composición)
cuenta_ahorro = CuentaBancaria(cbu="011002233", numero_tarjeta="4509-xxxx-xxxx-
1234")

# 3. Asociamos la cuenta al cliente (Agregación)
cliente_carlos.asociar_cuenta(cuenta_ahorro)

print(f"Cliente: {cliente_carlos.nombre}")
print(f"CBU asociado: {cliente_carlos.cuentas[0].cbu}")
print(f"Tarjeta emitida: {cliente_carlos.cuentas[0].tarjeta.numero}")
```